MEMORIA PROYECTO FINAL DE BASES DE DATOS

Gestión de Reviews mediante Bases de Datos

Lucía Raventós Gonzalvo y Manuel O'Shanahan Delgado-Taramona 2ºIMAT B

Introducción

A lo largo de este proyecto llevaremos a cabo una gestión completa de una Base de Datos la cual almacena múltiples reviews realizados por diferentes usuarios. Para ello, haremos una labor completa de almacenamiento de los datos (inicialmente en MySQL y MongoDB) y posteriormente un procesamiento de estos, generando una visualización analítica de los resultados. Esta será una aplicación interactiva que permite visualizar los datos a grandes rasgos e incluso especificar el comportamiento de los usuarios individualmente.

Por otro lado, también se gestionan todos estos datos mediante Neo4j, la cual es una aplicación de software que nos permite estructurar toda nuestra información como nodos y aristas, generando así una visualización mucho más llamativa y práctica.

Por último, se propone un modelo de aprendizaje automático capaz de realizar el mismo procesamiento que nosotros, e incluso ser más óptimo, estando capacitado de recomendar al usuario artículos que todavía no ha consumido pero que son bastante aclamados por el resto de los clientes.

Reparto del trabajo

Para este proyecto, hemos querido realizar un reparto bastante equitativo entre ambos miembros del equipo. Es decir, hemos tratado de dividir cada parte del proyecto de la mejor manera posible, y además cada integrante se ha hecho responsable de la parte de la memoria bajo su nombre, siempre teniendo también consciencia de la labor del otro compañero. El reparto quedaría algo así:

MANUEL

1. Primera parte: diseño y carga de datos en MySQL y MongoDB
2. De la tercera parte: 4.3 y 4.4
3. Cuarta parte.
4. Quinta parte: primero y tercero.

LUCÍA

1. Segunda parte: Aplicación Python para el acceso y visualización de los datos.
2. De la tercera parte: 4.1 y 4.2
3. Quinta parte: segundo
4. Opcional 8 (streamlit)

Primera Parte: Diseño y carga de datos

Para llevar a cabo este primer apartado de la práctica, lo más importante desde un principio ha sido analizar correctamente el contenido de cada uno de los ficheros JSON, especialmente teniendo en cuenta el tipo de dato almacenado en cada clave, y la forma en la que se pueden distribuir en las dos herramientas de gestión de Bases de Datos. Dado que el requisito inicial era distribuir la información en ambas aplicaciones, antes de normalizar cualquier tipo de relación es prioritario repartir los datos de manera idónea.

En primer lugar, para realizar este reparto, tuve muy en cuenta un concepto muy importante, y es que MySQL es una herramienta destinada a datos estructurados mientras que MongoDB suele almacenar datos no estructurados. Debido a esto, observando las variables contenidas en el JSON, he podido identificar que dos de ellas son cadenas de string bastante largas y "desordenadas", es decir, que no necesitan estar especialmente estructurados (ya que habrá reviews que podrán carecer tanto de un 'reviewText' como de un 'summary'). Por ello, esto dos atributos son los más oportunos a almacenar en MongoDB, ya que tampoco son valores numéricos o por los que vayamos a querer ordenar/agrupar nuestros datos. Nosotros, además de los atributos ya existentes, hemos querido añadir dos atributos fundamentales:

- **id_review:** esta variable es un contador que va a ir aumentando a medida que se almacene una review nueva en cualquiera de nuestras bases de datos. Es imprescindible que el id_review de MongoDB y MySQL sea el mismo para el mismo diccionario del JSON que esté leyendo, de esta manera podremos tener una relación entre ambos aplicativos.
- **categoria:** esta otra variable será una pequeña cadena string que contendrá el nombre del tipo de artículo al que le pertenece ese review. Es decir, como el contenido inicialmente se obtendrá de 4 ficheros diferentes con diferentes tipos de review, este string almacenará el nombre de esa categoría para posteriormente poder diferenciar los elementos.
  Lista de categorías:
  ["Video Games", "Toys and Games", "Digital Music", "Musical Instruments"]

En definitiva, en MongoDB crearemos una nueva base de datos, la cual nombraremos como 'comercio'. Dentro de esta base (que inicialmente se encuentra vacía), generaremos una colección denominada 'reviews'. Una vez hemos conseguido estipular una conexión-cliente con esa colección, iremos introduciendo uno a uno múltiples documentos, todos ellos con la siguiente estructura:

```
documento = {"_id": id_review,
             "categoria": categoria,
             "reviewText": reviewText,
             "summary": summary
             }
```

Por otro lado, hablando del diseño de datos en MySQL, el proceso de selección de almacenamiento de datos es bastante diferente, especialmente porque en este caso hemos tenido en cuenta la normalización de las relaciones. Para poder desarrollar mejor el proceso de normalización, primero es fundamental definir nuestro punto de partida. En MySQL también vamos a querer introducir los 2 atributos creados por nosotros mismos, es decir, tando 'id_review' como 'categoria'. Estos nos ayudan a facilitar los agrupamientos y las posibles conexiones con MongoDB. Otra gran duda que puede surgir es el por qué no hemos almacenado la variable 'helpful' dentro de MongoDB (al tratarse de una lista de valores). Esto se debe a que, como son valores numéricos y pueden servirnos para sumarlos o trabajar con ellos, hemos desplegado la variable 'helpful' en otras dos, una variable 'helpful_1' que contiene el primer valor de la lista helpful (en forma de número entero) y otra variable 'helpful_2' que contiene el segundo elemento de la lista (int).

Es decir, teniendo en cuenta todo esto, e imaginando que insertásemos todos los atributos en una sola relación de MySQL, tendríamos algo de esta manera:

reviews_inicial(<u>id_review</u>, categoria, reviewerID, asin, reviewerName, helpful_1,
helpful_2, overall, unixReviewTime, reviewTime)

Observando esta relación hipotética, se puede observar como hay una gran cantidad de dependencias funcionales entre atributos que vamos a analizar para llevar a cabo la normalización:

- id_review → categoria, reviewerID, asin, reviewerName, helpful_1, helpful_2, overall, unixReviewTime, reviewTime (**dependencias functionales completas)**
- reviewerID → reviewerName (**dependencia functional transitiva)**
- asin → categoria (**dependencia functional transitiva)**
- unixReviewTime → reviewTime (**dependencia functional transitiva)**

En primer lugar, la primera relación ya se encontraba en la primera forma normal, ya que todos los atributos son atómicos (al no ser compuestos ni multievaluados).

Además, sólo hay una clave primaria definida que en nuestro caso hemos querido que sea el id_review, ya que es el único atributo que no se va a repetir en ningún momento.

Por otro lado, la relación también se encuentra en la segunda forma normal, ya que la clave primara está compuesta por un único atributo, lo cual indica que es imposible que aparezca algún dependencia funcional parcial.

Por último, vemos que la relación NO se encuentra en la tercera forma normal, porque sí que se identifican ciertas dependencias funcionales transitivas. Por lo tanto, la forma de solucionar esto es generar una relación por cada dependencia transitiva e indiciar las claves foráneas en la relación con dependencia completa, como bien muestra nuestro esquema relacional final:

USUARIOS(reviewerID, reviewerName)
Primary Key: reviewerID

FECHAS(unixReviewTime, reviewTime)
Primary Key: unixReviewTime

ARTICULOS(asin, categoria)
Primary Key: asin

REVIEWS(<u>id_review</u>, reviewerID, asin, helpful_1, helpful_2, overall, unixReviewTime)
Primary Key: id_review
Foreign Key: reviewerID FK REFERS USUARIOS(reviewerID)
Foreign Key: asin FK REFERS ARTICULOS(asin)
Foreign Key: unixReviewTime FK REFERS FECHAS(unixReviewTime)

Como observación interesante, antes de crear este esquema tuvimos que evaluar realmente si debíamos premiar la optimización del programa o la normalización. Especialmente esta duda nos surgió frente a la relación de 'fechas', ya que es cierto que el valor en Unix y el tiempo en formato fecha es similar, solo que con diferentes representaciones. Pero aun así, hemos querido priorizar la normalización de nuestra base de datos, y más aún al darnos cuenta de que poner como clave primaria unixReviewTime reduce enormemente la cantidad de datos del tipo reviewTime a guardar, ya que para los casi 500000 datos almacenados, la relación de fechas tiene un número mucho menor de filas, de aproximadamente 5000 (especialmente porque diferentes reviewers han realizado reviews en el mismo momento y se repiten enormemente algunos de estos valores). Por último, cabe destacar que cada tipo de dato lo transcribimos al formato adecuado (ya sean strings, ints y en el caso de la fecha a formado DATE mediante la función strptime).

Segunda Parte: Aplicación para visualización

En la segunda parte del proyecto se ha desarrollado una aplicación de visualización para representar de forma gráfica los datos almacenados en la base de datos. El objetivo es facilitar su análisis e interpretación mediante una interfaz interactiva que permite realizar consultas y mostrar los resultados de forma clara y sencilla.

1. DISEÑO DE LA APLICACIÓN

Para montar el diseño de esta segunda parte se ha diseñado una aplicación interactiva de visualización de datos, cuyo objetivo es permitir la exploración gráfica de la información almacenada en la base de datos. Esta aplicación se ha desarrollado en Python usando la librería Streamlit, y Matplotlib para la generación de gráficos, que es con la que más familiarizados estamos. Es decir, para no trabajar con la comunicación con terminal y un display de gráficos puntuales, hemos querido implementar desde un principio el último opcional en este apartado, generando un aplicativo (en forma de página web) capaz de hacer un análisis prácticamente completo de los datos.

La interfaz se ha organizado mediante un menú lateral desde el que se selecciona la visualización que se desea mostrar. A su vez, cada gráfico presenta dentro de la propia página sus controles específicos, como selectores de categorías o filtros. Esta decisión permite evitar una interfaz sobrecargada y mejora la claridad, ya que cada pantalla muestra solo las opciones necesarias para la visualización de ese momento.

2. SOFTWARE DE LA APLICACIÓN

Al ejecutar el fichero *load_data.py*, se han cargado los datos en las aplicaciones MySQL y MongoDB, cada una con sus respectivos formatos decididos en la primera parte. De esta forma, el software de la aplicación se conecta directamente a la base de datos en ambas aplicaciones y realiza las consultas necesarias sobre ella.

A nivel de implementación, cada gráfico se ha desarrollado como una función independiente. Esto permite estructurar el código de manera modular, separando la lógica de consulta, el tratamiento de resultados y la representación gráfica. Además, esta organización facilita el mantenimiento y la ampliación de la aplicación, ya que resulta sencillo incorporar nuevas visualizaciones en el futuro.

3. CONSULTAS

En esta segunda parte, las consultas se han planteado para obtener de la base de datos la información necesaria para cada una de las visualizaciones de la aplicación. La idea seguida ha sido que cada gráfico tenga asociada su propia consulta, de forma que al seleccionar una opción en la interfaz se recuperan únicamente los datos necesarios para esa representación. Esto hace que la aplicación sea más clara, más ordenada y fácil de mantener.

En general, las consultas se basan en operaciones de filtrado, agrupación y recuento. Gracias a ello, se puede representar la evolución de las reviews por años, la popularidad de los artículos, la distribución de notas o la actividad de los usuarios.

En la última consulta, se ha optado por una visualización centrada en los usuarios. En este caso, la aplicación pide uno o varios identificadores y comprueba primero cuáles existen realmente en la base de datos. Una vez validados, para cada usuario se obtiene su distribución de notas, es decir, cuántas veces ha dado cada puntuación. Además, se calculan algunas métricas sencillas, como el número total de reviews realizadas o su nota media.

A partir de esa información, la aplicación muestra un pequeño resumen individual de cada usuario, lo que permite ver de manera rápida su forma de valorar los productos.

Si se introducen varios identificadores, también se realiza una comparación conjunta, mostrando una tabla comparativa y un gráfico donde se aprecian mejor las diferencias entre usuarios.

Así, esta consulta no solo permite ver datos de forma aislada, sino también comparar comportamientos y detectar si unos usuarios tienden a puntuar más alto o más bajo que otros.

Con esta forma de plantear las consultas, la aplicación no solo sirve para mostrar gráficos, sino también para facilitar el análisis de la información almacenada en la base de datos de una manera más visual y sencilla.

4. PRÁCTICO

Tratando en mayor profundidad cada consulta, en este caso el fichero menu_visualizacion.py no se puede ejecutar como cualquier otro, ya que, al tratarse de una aplicación de streamlit, es fundamental que el usuario se ubique en la ruta de la carpeta en su ordenador, y preferiblemente ejecute el siguiente comando:

```
python -m streamlit run
.\proyecto_final_bases_datos\menu_visualizacion.py
```

Una vez se ejecuta esa línea, la terminal debería devolver algo pareció a lo mostrado en la **Figura 1** (siempre y cuando las credenciales en configuración.py se hayan adaptado al usuario en cuestión).

```
You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://192.168.1.34:8501
```

Figura 1. Terminal al ejecutar con streamlit el menu_visualizacion.py, indica la URL de la página web.

Es decir, una vez encontrado ese mensaje, si se hace CTRL + Click sobre el primer http (URL), debería dirigir al usuario directamente hacia nuestro aplicativo. Una vez se abre el link en su navegador, la página principal debe ser similar a lo que se representa en la **Figura 2**. En esta, se puede identificar tanto el título del proyecto, como a la izquierda un menú interactivo que contiene todas las opciones y consultas posibles.

Figura 2. Página de inicio de la aplicación y de la barra de elección de consultas.

Haciendo un breve repaso para demostrar la presencia de todas las consultas, por ejemplo, si se pulsa en el botón de 'Reviews por años', obtendríamos el gráfico que muestra la cantidad de reviews realizados cada año de todos los productos o de una categoría en concreto, como se trata de mostrar en la **Figuras 3 y 4**. En esta, como bien se observa, hay un menú deslizante que le permite al usuario seleccionar la categoría por la que realizar la búsqueda, o, por el contrario, tener en cuenta todos los datos almacenados.

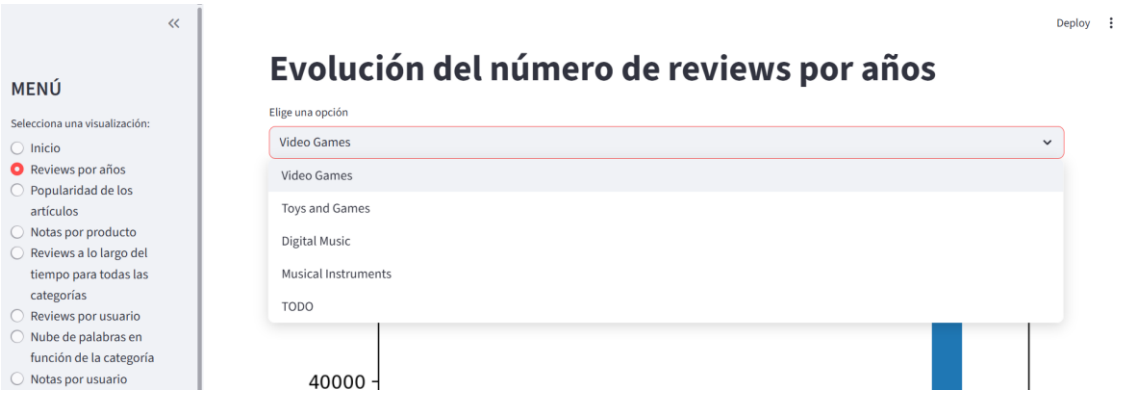


Figura 3. Pestaña de "Evolución del número de reviews por años" y sus opciones

Figura 4. Histograma que representa la evolución del número de reviews por años de los productos de todas las categorías.

Como ejemplo, en la visualización de la **Figura 4** , el usuario puede seleccionar una categoría concreta o representar el conjunto total de datos. En función de esta selección, se ejecuta una consulta SQL distinta, con o sin filtro WHERE, y los resultados se muestran mediante un gráfico de barras, todo esto contenido en la función evolución_por_años del fichero Este tipo de representación se ha

considerado el más adecuado, puesto que los datos se agrupan por años y, por tanto, son discretos.

De esta manera se desenvuelven el resto de las consultas, cada una comunicándose con MySQL especialmente para buscar los datos pertinentes y trabaja con Python y Matplotlib para poder representar los resultados visualmente. Para el segundo apartado, por ejemplo, nos permite de nuevo seleccionar una de las categorías y nos devuelve un gráfico como el de la **Figura 5**.

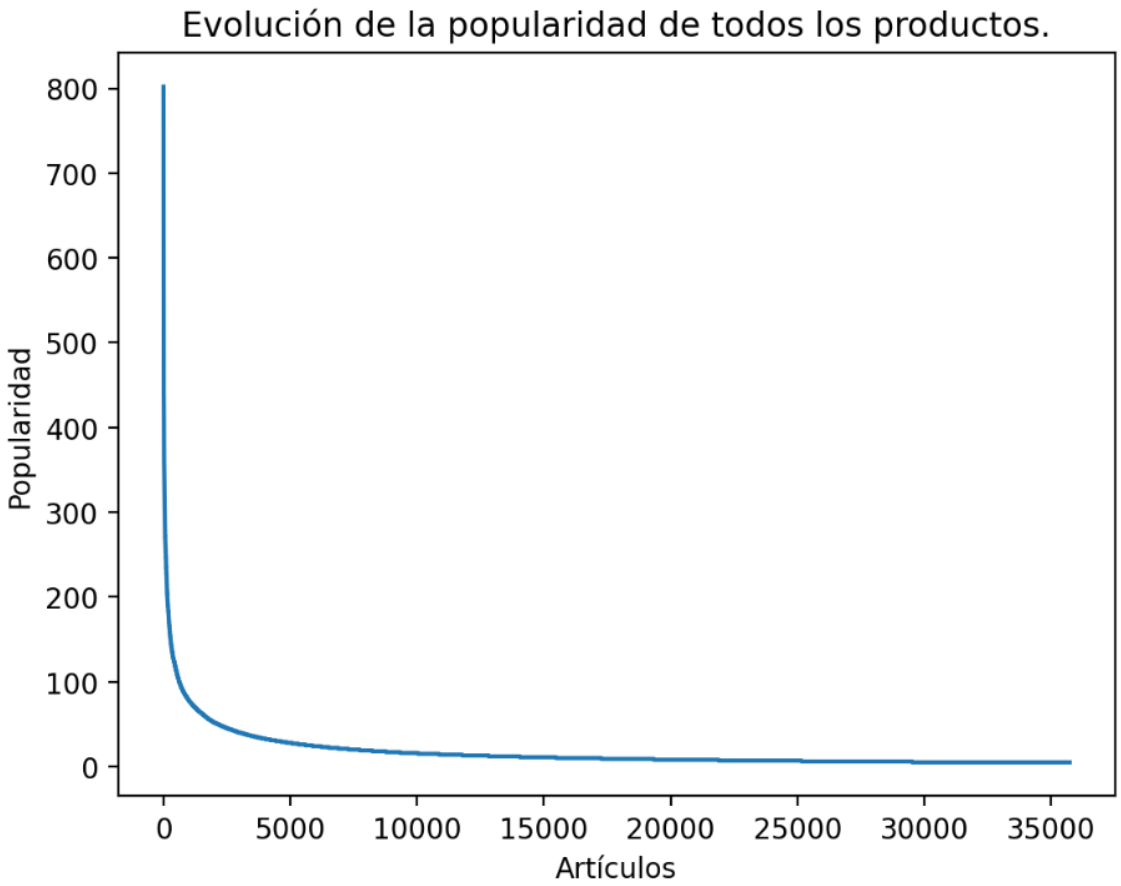


Figura 5. Representación desde streamlit de la evolución de la popularidad de los artículos teniendo en cuenta TODOS los productos.

Se invita al usuario a navegar libremente por todo el aplicativo, comprobando que todas las consultas se encuentran representadas. Cabe destacar que, en alguna de ellas, como, por ejemplo en la de reviews por usuario, hemos añadido nuestra pequeña mejora procurando mostrar el gráfico en escala normal y en escala logarítmica, ya que consideramos que esta representa mucho mejor la realidad y permite distinguir la densidad de los datos con mayor facilidad.

Por último, consideramos oportuno realizar una pequeña explicación visual de la forma en la que funciona nuestro “análisis opcional”, ya que contiene muchos plots y es fundamental la forma en la que se introduce a los usuarios. Como bien

comentamos, se puede introducir un solo reviewerID, o en el caso de querer comparar las notas dadas por varios usuarios, se debe hacer separándolos por comas, como se muestra en la **Figura 6**.

Figura 6. Menú de 'Notas por usuario' desplegado y funcionando correctamente.

Una vez se introducen dos IDs, podemos observar cómo se despliega la página y aparece un análisis muy detallado tanto del conjunto como de cada uno individualmente. Primero muestra un histograma con la media de los dos, posteriormente los histogramas de cada uno individualmente, y es entonces cuando realiza mediante una tabla una distinción entre ambos y comienzan a aparecer histogramas comparativos, como se muestra en la **Figura 7**.

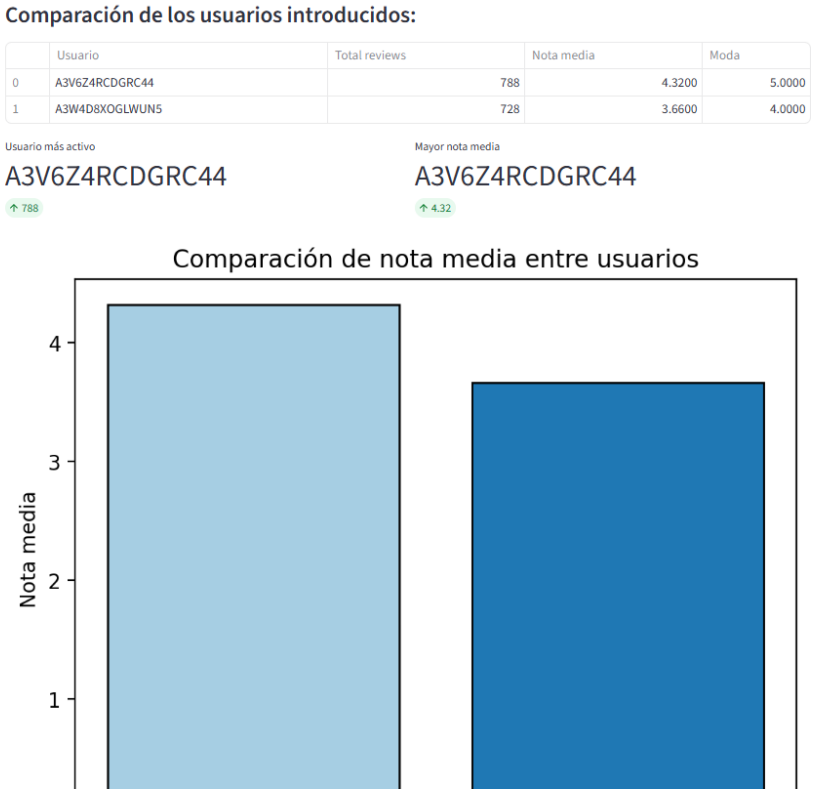


Figura 7. Análisis de dos usuarios concretos (A3V6Z4RCDGRC44 y A3W4D8XOGLWUN5) en el menú de 'Notas por usuario'.

Además, se puede observar que hay una opción en el menú más, reconocida como 'Artículos populares no consumidos', pero de esa función se hablará más adelante, ya que es una sección de la QUINTA PARTE del proyecto que hemos querido introducir en nuestro aplicativo de streamlit para que fuese más visual.

Por último, también contamos con una opción de SALIDA, que realmente nos indica cómo proceder para acabar con el programa (sería cerrar la pestaña del navegador y pulsar CTRL + C en la terminal de la **Figura 1**). No obstante, aunque no sea la forma más óptima, hemos querido añadir una pequeña implementación en la que, al pulsar el botón, mediante la librería 'os' de Python y usando las señales mediante 'signal' que hemos visto en la asignatura de FUSO, lo que estamos haciendo es matar el proceso de la terminal, por lo que es una forma de realizar el CTRL + C sin obligar al usuario a volver a comunicarse con la terminal directamente.

En conclusión, el diseño de esta segunda parte se ha orientado a construir una herramienta sencilla, funcional e intuitiva, capaz de aprovechar la base de datos desarrollada previamente y ofrecer una forma visual de analizar su contenido.

Tercera Parte: Aplicación Python con Neo4j

Para llevar a cabo este apartado, se ha realizado un fichero neo4JProyecto.py que contiene todas las funciones y el main necesarios para poder ejecutar sin problema alguno todo este apartado. En resumidas cuentas, hemos procurado estructurar las funciones lo mejor posible, aunque todo el código se encuentra repartido en bastantes funciones y para cada funcionalidad existe una función que se encarga de llamar al resto y agrupar toda la información.

En nuestro caso, el menú main consiste en un pequeño menú ejecutado en la terminal que permite seleccionar una de las consultas/funcionalidades de este apartado, y en algunos casos con ciertas excepciones que se irán visualizando a medida que abarquemos cada apartado. Por lo tanto, para observar los resultados de esta parte, basta con ejecutar el fichero en una terminal mediante Python.

1. Obtener similitudes entre usuarios y mostrar los enlaces en Neo4J

Para el primer apartado, es posiblemente uno de los más complejos, ya que hemos tenido que calcular la correlación de Pearson para observar la similitud entre todos los participantes. Antes de nada, cabe destacar que el número de usuarios con los que trabajar se puede definir directamente desde configuración.py, aunque como valor predeterminado se encuentra 30 (se adapta a cualquier número). Por ejemplo, para este apartado, hemos querido crear un índice para acelerar la consulta y posteriormente eliminarlo, al igual que hemos decidido imponer ciertas restricciones a los nodos de Neo4j, para que por ejemplo no existan nodos de usuario iguales, y así observar correctamente las similitudes entre ellos. Cabe destacar que el proceso de carga de datos lo hace desde la propia RAM, y que mediante un bucle va insertando uno a uno cada nodo y sus relaciones pertinentes (después de haber vaciado la base de datos por completo). Observando los resultados una vez termina la carga, podemos observar que para este caso se terminan creando 30 usuarios y 402 relaciones de similitud, como se muestra en la **Figura 8**.

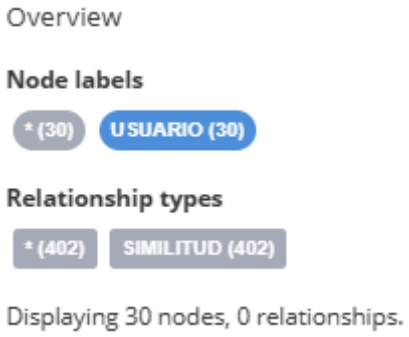


Figura 8. Leyenda del mapa de los nodos y las relaciones entre sí con N=30.

Si queremos fijarnos en la visualización del grafo completa, aunque sea muy complejo discernir las similitudes entre sí, podemos confirmar que obtenemos un grafo muy similar al de la **Figura 9**, y viendo como un nodo no tiene vecinos directos (pero es representado al tener de las 30 mayores cantidades de reviews).

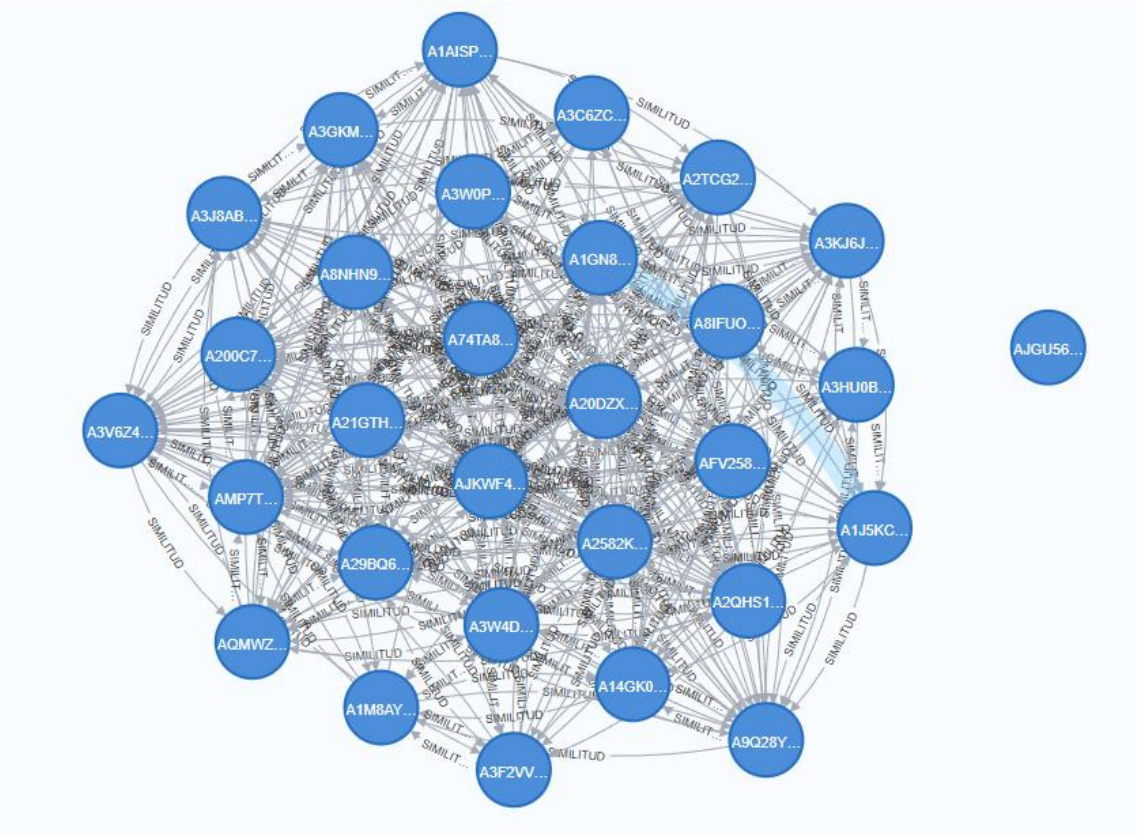


Figura 9. Captura de pantalla de la representación de los nodos USUARIO y sus relaciones bidimensionales SIMILITUDES mediante la aplicación Neo4J.

Además, la propia terminal de Python, al terminar de cargar los datos en Neo4j hace una pequeña consulta para visualizar quiénes son el nodo/los nodos con el mayor número de vecinos, y en nuestro caso nos devuelve algo como lo de la siguiente imagen (dos nodos al haber empate):

```
Usuario(s) con el número máximo de vecinos:
Usuario: AJKWF4W7QD4NS | Número de vecinos: 23
Usuario: A20DZX38KRBIT8 | Número de vecinos: 23
```

Figura 10. Función mostrar_usuario() por terminal, devolviendo al user con más vecinos:

Siendo igual que en la aplicación si se visualiza en formato de tabla como la **Figura 11,** ordenando los nodos por mayor número de vecinos a menor:

	u	vecinos
1	A20DZX38KRBIT8	23
2	AJKWF4W7QD4NS	23
3	A2582KMXLK2P06	22
4	A3W4D8XOGLWUN5	22
5	AMP7TQRWAIE84	22
6	A3V6Z4RCDGRC44	18
7	A1AISPOIIHTHXX	17
8	A2QHS1ZCIQOL7E	16

Figura 11. Tabla generada por Neo4j que muestra el id del reviewer y a su lado el número de vecinos que tiene

2. Obtener enlaces entre usuarios y artículos.

En segundo lugar, tenemos esta otra consulta la cual es un poco peculiar, ya que en primer lugar realiza una distinción según la categoría de la que se trate. Además, se le permite seleccionar un número de artículos aleatorios que mostrar al usuario en cuestión. Es decir, primero nos muestra las categorías disponibles, y no se permite avanzar si no se inserta algunas de las mostradas como opción, y por otro lado hay que insertar un número entero a observar. Por ejemplo, haciendo la consulta para 30 artículos y la categoría de 'Video Games', la terminal nos avisará cuando los datos estén cargados y podremos acceder a la aplicación de Neo4j para observar los resultados de la **Figura 12**.

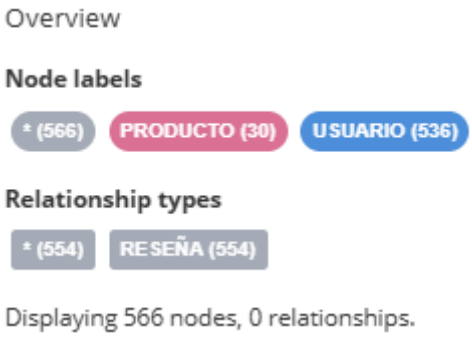


Figura 12. Leyenda del mapa de los nodos y las relaciones entre sí.

Si nos fijamos, en este caso nos devuelve los 30 productos solicitados, pero a su vez nos muestran los 536 relacionados con esos productos. Por otro lado, se puede observar que hay 554 reseñas, es decir, relaciones entre los diferentes nodos, y todo esto se observa mejor en la **Figura 13**.

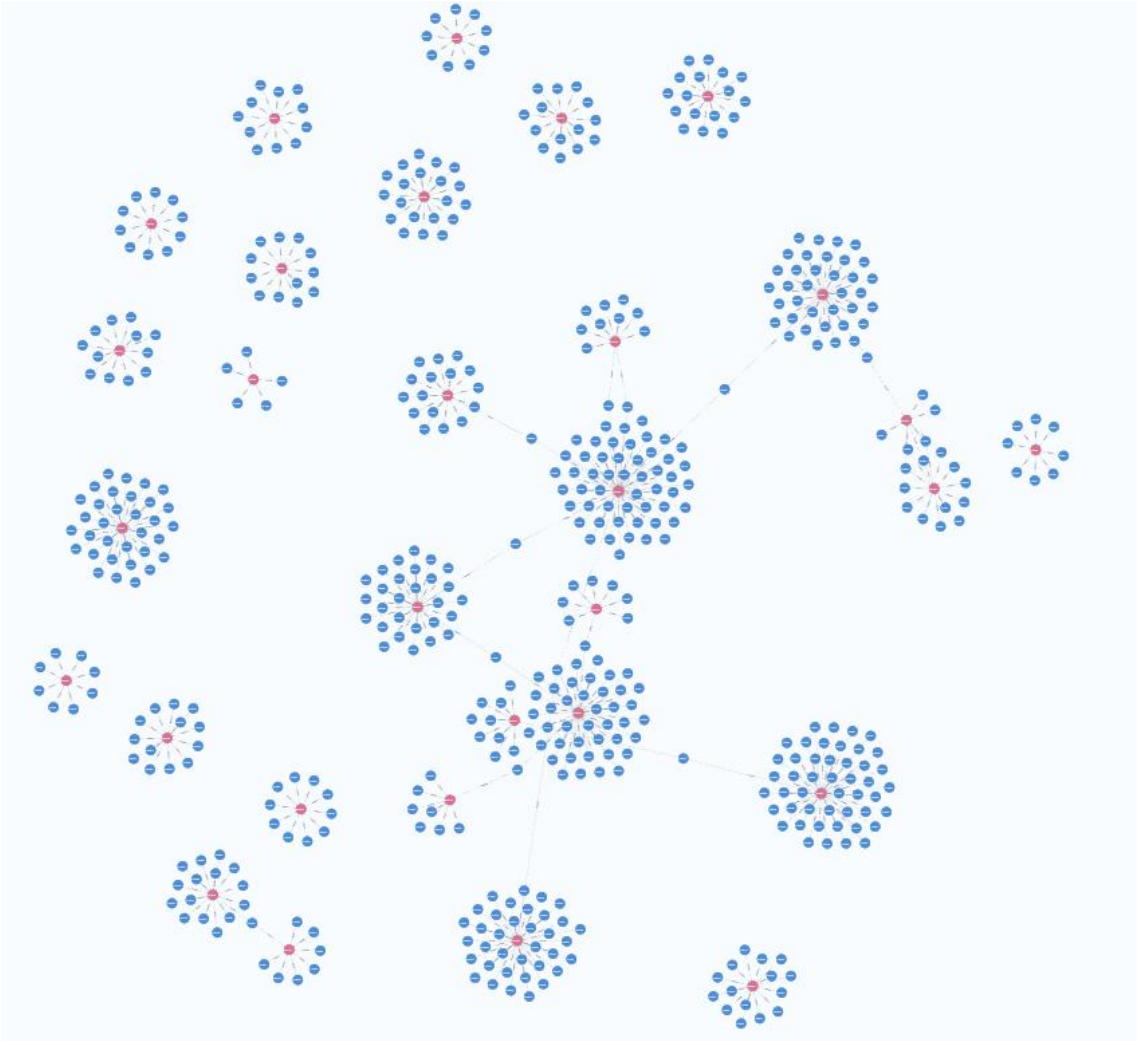


Figura 13. Captura de pantalla de la representación de los nodos PRODUCTO y USUARIOS.

De igual manera, es altamente interesante observar cómo cambian los resultados a la hora de cambiar tanto la categoría como el número de artículos.

3. Obtener usuarios que hay visto más de un tipo de artículo

Para llevar a cabo este tercer apartado, hemos llevado a cabo una estructura que para el resto de casos, teniendo muy en cuenta las diferentes consultas a realizar, las inserciones pertinentes y juntándolo todo en la función de tercera_funcionalidad(). No obstante, a este ejercicio hemos querido añadirle un pequeño extra, ya que, se nos pedía que ordenásemos los nombres de los reviewers

(por inducción hemos entendido que se refería a ordenarlos ascendente), y al realizar esta búsqueda, nos hemos percatado de que en esos 400 primeros usuarios CASI TODOS tenían un nombre nulo (351 filas de las 400). Es decir, como al ordenar ascendentemente primero pone a los nulos, estaba teniendo en cuenta todos estos usuarios. Por lo tanto, hemos querido darle la oportunidad al cliente de que decida si, quiere realizar la consulta teniendo en cuenta los nulos y ordenando el desempate por el reviewerID de manera ascendente, o si por el contrario prefiere descartar los nulos y ordenar por nombres "medianamente válidos". Por lo tanto, esta consulta se encarga de buscar a esos 400 usuarios, observar cuántos han puntuado ítems de categorías distintas, y además tener en cuenta cuántos ítems de cada categoría han evaluado. Si nos mantenemos con la opción 1 (descartar todos los nulos) obtenemos un esquema de los resultados como el de la **Figura 14**, junto a su grafo a su lado.

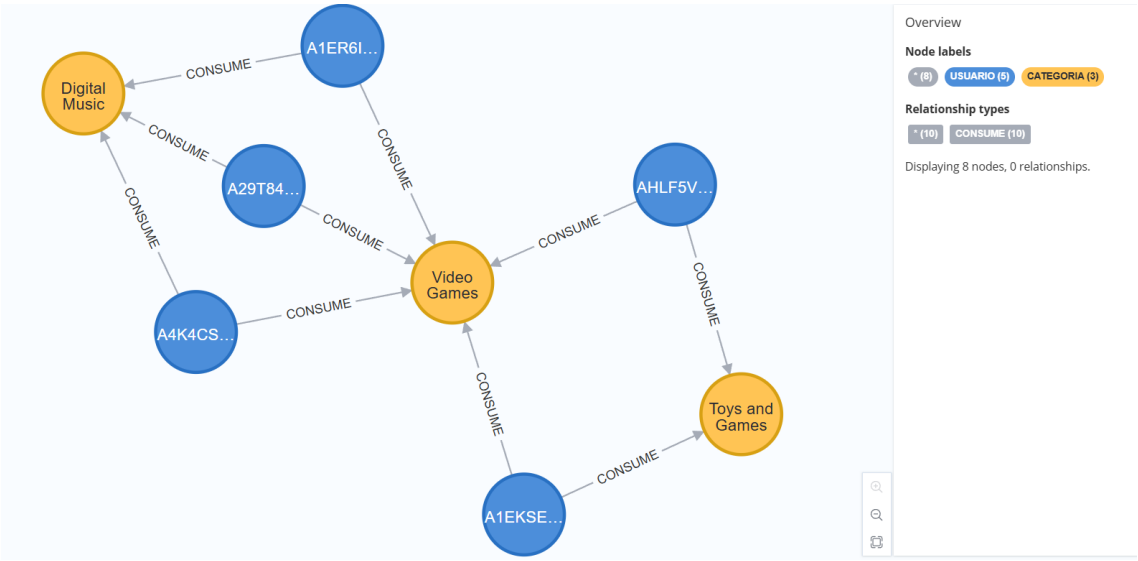


Figura 14. Grafo de la consulta del apartado 3 (descartando los nombres nulos) junto con su resumen

Observando la imagen podemos observar que de los 400 usuarios por los que hemos filtrado, únicamente 5 de ellos han realizado reviews a dos categorías distintas. Es más, ninguno ha realizado ningún review a las 3 categorías, lo cual también muestra que ninguno de esos usuarios ha puntuado un artículo de la categoría 'Musical Instruments'. Esto sin duda es una conclusión bastante curiosa, porque muestra como los usuarios sin nombres nulos parecen hacer reviews de sólo una categoría (o al menos los ordenados por nombre ascendente).

Por otro lado, si nos centramos en la opción 2, en la cual se tiene en cuenta los nulos y se desordena por desempate (realmente da un poco igual porque hay menos nulos que el límite de la consulta), aún así vemos en la **Figura 15** que el resultado ha cambiado exponencialmente, y aquí sí se muestran muchos más usuarios.

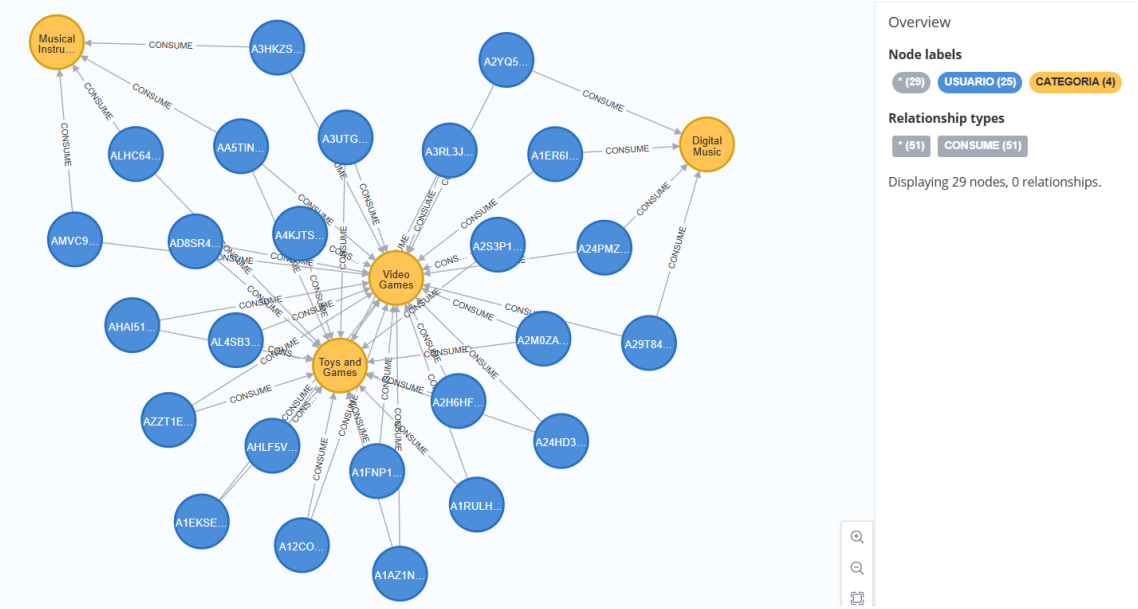


Figura 15. Grafo de la consulta del apartado 3 (teniendo en cuenta los nombres nulos) junto con su resumen

Esta pequeña distinción nos muestra lo importante que es observar los datos que se nos proporcionan, y como modificando una simple línea de la consulta aparecen 20 usuarios más y las 4 categorías por fin se ven representadas.

4. Artículos populares y en común con usuarios

Por último, este apartado ya podemos deducir un poco el resultado antes de realizarlo, ya que, en el apartado 2, vimos la densidad de artículos según su número de reviews, y si recordamos, para los primeros 500 artículos, estos tenían una densidad altísima. Es decir, somos conscientes que habrá muchísimos artículos con menos de 40 reviews (porque además es lo habitual). Al quedarnos solo con los 5 más populares, todos tendrán 39 reviews, lo que cual indica que es posible como máximo que hay 39 * 5 usuarios que les hayan evaluado. De igual manera, todo este análisis se ha realizado con Python, y sobre todo las consultas pertinentes para terminar obteniendo el siguiente esquema:

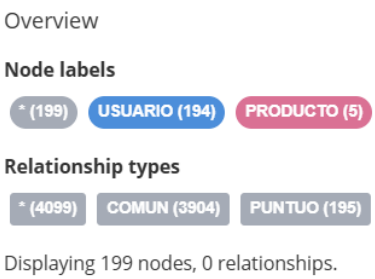


Figura 16. Esquema de los resultados de los artículos y los usuarios que les han puntuado

Si nos fijamos, la observación mas curiosa es que justamente 39 * 5 = 195, y el esquema nos ha mostrado que se han creado 194 usuarios con 195 relaciones de puntuo, sin embargo, lo que más abunda son las relaciones de común entre ellos, demostrando como sobre todo habrán puntuado en común artículos con más de 40 reviews acumulados. Todas estas conexiones entre sí se visualizan especialmente bien en la **Figura 17**.

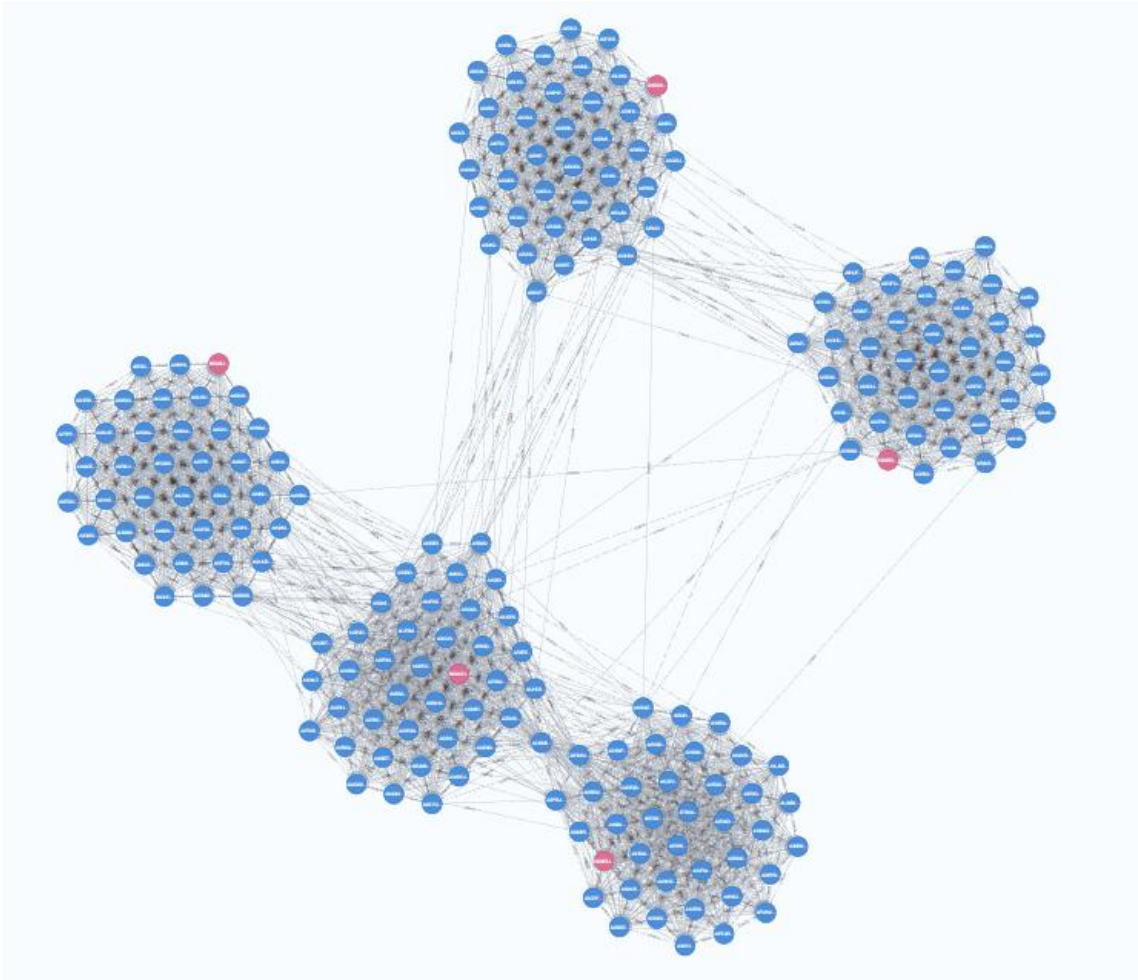


Figura 17. Representación visual de los 5 artículos con menos de 40 reviews y todos los usuarios que los han puntuado.

Cuarta Parte: Nuevos datos

De cara a insertar nuevos datos en nuestras bases de datos ya generadas, simplemente hemos respetado el modelo normalizado ya definido anteriormente. Es decir, hemos tratado de adaptar el fichero inserta_dataset.py para que gestione cualquier fichero nuevo que se quiera introducir, porque con tal de que mantenga el mismo formato que el resto de JSON, debería ser capaz de almacenarlo (siempre que también se indique el valor de la categoría (str)). Cabe destacar que en este apartado nos surgió una pequeña duda interna, y es que es cierto que es bastante complejo (y ocupa mucha memoria) almacenar todas las claves primarias de las relaciones pequeñas para evitar que se repitan. Es decir, en un principio pensamos en utilizar el comando de MySQL 'INSERT IGNORE', de tal manera que si aparecía por ejemplo un reviewerID que ya estuviese almacenados en la relación de 'usuarios', no saltase un error y simplemente no lo introdujese. El problema es que, como esta función también ignora cualquier tipo de error, podríamos sin querer romper la arquitectura de datos estructurados que representa a este aplicativo. Es por eso que hemos creado una función que captura el último índice apuntado para seguir desde ese valor, y otra función que recupera todas las claves primarias de las relaciones y las guarda en un set para poder realizar las inserciones y asegurarnos de que en ningún momento se repetirán.

Por último, y para poner a prueba este fichero, hemos querido insertar los datos de un JSON llamado 'Grocery_and_Gourmet_Food_5.json', el cual contiene información sobre alimentación, pero lo único que hay que modificar es que, en lista rutas, también se debe indicar el nombre de la categoría, y de resto el código se encarga de funcionar por sí solo.

Quinta Parte: Más visualización y Machine Learning

APARTADO 1

En este apartado se solicitaba replicar las gráficas de la segunda parte del proyecto en un aplicativo que permitiese hacer DashBoards. En nuestro caso, al principio nos costó bastante elegir uno, hasta que informándonos descubrimos que PowerBi Desktop (la función que tenemos instalada) nos daba la posibilidad de conectarnos a una base de datos de MySQL para trabajar con los datos almacenados desde ahí, y así no nos obligaba a pasar la información a otro formato (como podría ser un csv). Es cierto que para ello hemos tenido que instalarnos un comunicador, conocido como un mysql_connector, el cual genera la conexión entre ambas aplicaciones y permite recoger todos los datos, pero sin duda ha sido primordial porque también mantuvo el formato fecha en todo momento y sobre todo facilitó enormemente la representación visual de los gráficos. En nuestro caso, hemos querido replicar el gráfico del apartado 1 de esa sección, el apartado 3 y por último (y el más complicado) el apartado 4. Digo que este ha sido el más complejo porque nos ha obligado a crear una función a parte que calculase la media acumulativa para poder representarla en función del tiempo, pero finalmente todo ha salido increíblemente bien.

Cabe destacar que el propio DashBoard es interactivo, ya que permite cambiar los gráficos según la categoría que se seleccione, e incluso cambiar los intervalos de años en los que se analizan los datos. Desafortunadamente sólo podemos adjuntar una captura de pantalla del resultado final, pero adjuntamos aquí también una URL que debería permitir a cualquier persona visualizar el resultado final:

https://app.powerbi.com/view?r=eyJrIjoiZjhlZmY1YjgtZGI5OS00MzJiLWE3OTktMzAyOWM1OWUzMTAyIiwidCI6ImJjZDI3MDFjLWFhOWItNGQxMi1iYTIwLWYzZTNiODMwNzBjMSIsImMiOjh9

Finalmente, aunque se pueda visualizar en ese link, el resultado final sería el de la **Figura 18.**

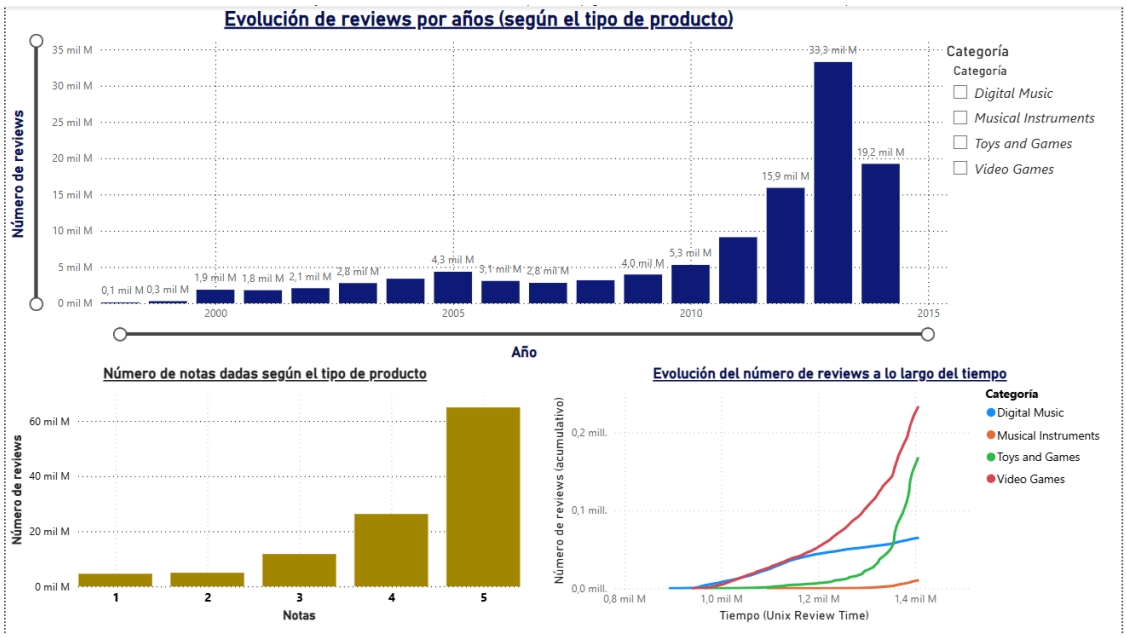


Figura 18. DashBoard en PowerBi del apartado 1 de la quinta parte

Diseño de modelo de Machine Learning

Para diseñar un modelo de Machine Learning con los datos almacenados, seguiríamos una serie de instrucciones necesarias para poder generarlo.

Típicamente, en primer lugar, se realiza una exploración inicial, junto a un análisis descriptivo del conjunto de datos (EDA). Por suerte, ya conocemos las variables que contiene la base de datos debido a que hemos tratado anteriormente con ella. Por tanto, este paso sería rápido.

En segundo lugar, nos dispondríamos a realizar una limpieza de los datos para mejorar la calidad del espacio de entrada. El preprocesamiento consiste en:

a) Lidiar con 'missing values' / valores faltantes.
   Según la cantidad de datos que tengamos no nulos, los conservaremos o los eliminaremos de la base. En el caso de decidir conservar alguno, bucaríamos un valor adecuado para esa variable mediante algún método de imputación como 'KNN imputation'. Estamos familiarizados con ellos ya que los vimos en la asignatura de Adquisición de Datos.
   Cabe destacar que en nuestro dataset, este paso ya se ha realizado anteriormente (nos lo han dado hecho), pero nunca está mal asegurarse.

b) Graficar los datos y realizar una búsqueda de 'outliers' / datos atípicos.

Utilizando diferentes visualizaciones como ‘Box-plots’ y ‘Qq-plots’, se observan fácilmente. Al identificarlos, hay que revisar de qué tipo son: errores de carga, errores de medición o casos reales pero raros. Si son errores, se deben corregir, por ejemplo, marcándolo como faltante y posteriormente, aplicar imputación, o eliminar. En el caso en el que sean casos reales se deben seguir almacenando a pesar de que sea extraño o muy distinto a los demás.

Nuestro dataset contiene variables en las que tiene sentido hablar de outliers y otras en las que no. Por ejemplo: ‘helpful’ representa algo como votos útiles/votos totales; un posible valor atípico podría ser proporciones raras como [0,100]. En cambio, en la variable ‘overall’, que es la puntuación asignada en una reviews, no hay posibilidad de outlier debido a que esta acotada y sus valores extremos (1 o 5) son válidos.

c) Codificar variables categóricas.

Como ya sabemos, los ordenadores no entienden de palabras, únicamente de números. Por ello, las variables categóricas se deben transformar a numéricas, y esto se hace mediante el método *‘one-hot encoding’* o ‘*dummy encoding*’.

En nuestra base de datos, este punto es muy importante. La mayoría de variables son categóricas (‘reviewerID’, ‘reviewerName’, ‘summary’...) y por tanto, deberán realizarse las transformaciones correspondientes en dichas variables.

d) Analizar las variables continuas.

El análisis de variables continuas debe consistir, principalmente, en estandarizarlas, quitar la asimetría con una transformación Box-Cox cuando están muy sesgadas y gestionar la colinealidad con reducción de dimensionalidad. Estos pasos suelen mejorar la estabilidad, el rendimiento y la interpretación estadística, aunque no todos son necesarios en todos los modelos.

Este paso es importante debido a que si no estandarizas, las variables con valores numéricos más grandes pueden dominar el modelo, aunque no tengan más información importante. Asimismo, la colinealidad aparece cuando varias variables explicativas están muy correlacionadas entre sí y esto genera problemas como coeficientes inestables o peor generalización en algunos modelos. Por eso, es importante tratarla.

La transformación Box-Cox ayuda a acercar la distribución a una forma más simétrica y reducir heterocedasticidad. En general, estos cambios se

realizan porque ciertos modelos funcionan mejor cuando las variables tienen escalas comparables, las distribuciones no están tan sesgadas y los predictores no son excesivamente redundantes.

En el dataset que estamos tratando, este paso es irrelevante debido a que la única variable numérica que no es discreta es 'unixReviewTime'. Sin embargo al representar tiempo, suele tratarse como temporal, no como continua pura.

e) Dividir los datos.
Este paso es uno de los más importantes al crear un modelo de Machine Learning. Para poder predecir correctamente sobre datos nuevos, es necesario dividir el conjunto de datos en, al menos, dos partes: un conjunto de entrenamiento (*train*), con el que se ajusta el modelo, y un conjunto de prueba (*test*), con el que se evalúa su rendimiento. Y si es posible, realizar *cross-validation* separando el conjunto de *train* en *train* y *validation* para realizar un estudio mejor de los parámetros.
Si el modelo se evalúa con los mismos datos con los que fue entrenado, el error obtenido será demasiado bajo y no reflejará su capacidad real de generalización. Esto ocurre porque el modelo ya ha visto esos datos y puede ajustarse excesivamente a ellos. Como consecuencia, al aplicarlo a observaciones nuevas, su rendimiento suele empeorar.

Por ello, en nuestro modelo será necesario dividir adecuadamente el conjunto de datos antes del entrenamiento, para obtener una evaluación más realista y reducir el riesgo de sobreajuste (*overfitting*).

Una vez preprocesados los datos, el siguiente paso consiste en diseñar un sistema de recomendación capaz de sugerir productos a usuarios que no hayan interactuado previamente con ellos.

Para diseñarlo, personalmente propondría usar el algoritmo 'K-Nearest Neighbors', que realiza una aproximación no paramétrica del clasificador de Bayes. Este modelo parte de la hipótesis de que usuarios con valoraciones similares en artículos comunes tienden a tener gustos parecidos, por lo que los productos bien valorados por usuarios similares pueden ser recomendados al usuario objetivo.

La idea informal podría ser la siguiente: cada usuario queda representado por las valoraciones que ha dado a distintos productos. A continuación, se compara un

usuario con el resto, se buscan los usuarios más parecidos y se recomiendan productos que esos usuarios similares hayan valorado bien y que el usuario objetivo todavía no haya consumido.

Para construir el modelo, se utilizarían principalmente las variables 'reviewerID', que identifica al usuario, 'asin', que identifica el producto, y 'overall', que representa la preferencia explícita del usuario. Asimismo, para mejorar o complementar el sistema podría tener también en cuenta las variables: reviewText, que nos da la opinión del usuario sobre el producto, summary, que nos resume la opinión del usuario sobre el producto, unixReviewTime / reviewTime, que permitirían dar más peso a reviews recientes, o helpful, que podría utilizarse para ponderar más las reseñas consideradas útiles.

Con las principales se genera una matriz usuario-artículo en la que cada fila representa un usuario, cada columna un producto y cada celda la puntuación asignada por dicho usuario a ese producto. Con las variables complementarias, se podrían hacer otras matrices para complementar la suposición .

A partir de esa matriz, se calcula la similitud entre usuarios. Como medida de similitud se puede emplear la correlación de Pearson, que además de que la hemos usado anteriormente, permite comparar el patrón de valoraciones entre dos usuarios teniendo en cuenta sus diferencias de escala al puntuar. Es decir, si por ejemplo, un usuario es más crítico, la nota que otorgara a un producto que le haya gustado será entre un 3 o 4, mientras que un usuario más agradecido será un 5, y esto lo tiene en cuenta la forma que hemos elegido para calcular la similitud entre dos usuarios.

Una vez calculadas las similitudes, para cada usuario se seleccionan sus k vecinos más próximos, es decir, los usuarios con gustos más parecidos. Después, para recomendar artículos no consumidos, se buscan productos que esos vecinos hayan puntuado positivamente y que el usuario todavía no haya valorado. Finalmente, los artículos candidatos se ordenan según una puntuación estimada y se recomiendan los N mejores.

Por último, antes de sacar el modelo a la luz, se debe evaluar con una función discriminante. Para ello, mediante el conjunto de 'test', que hemos dividido y guardado anteriormente, se prueba.

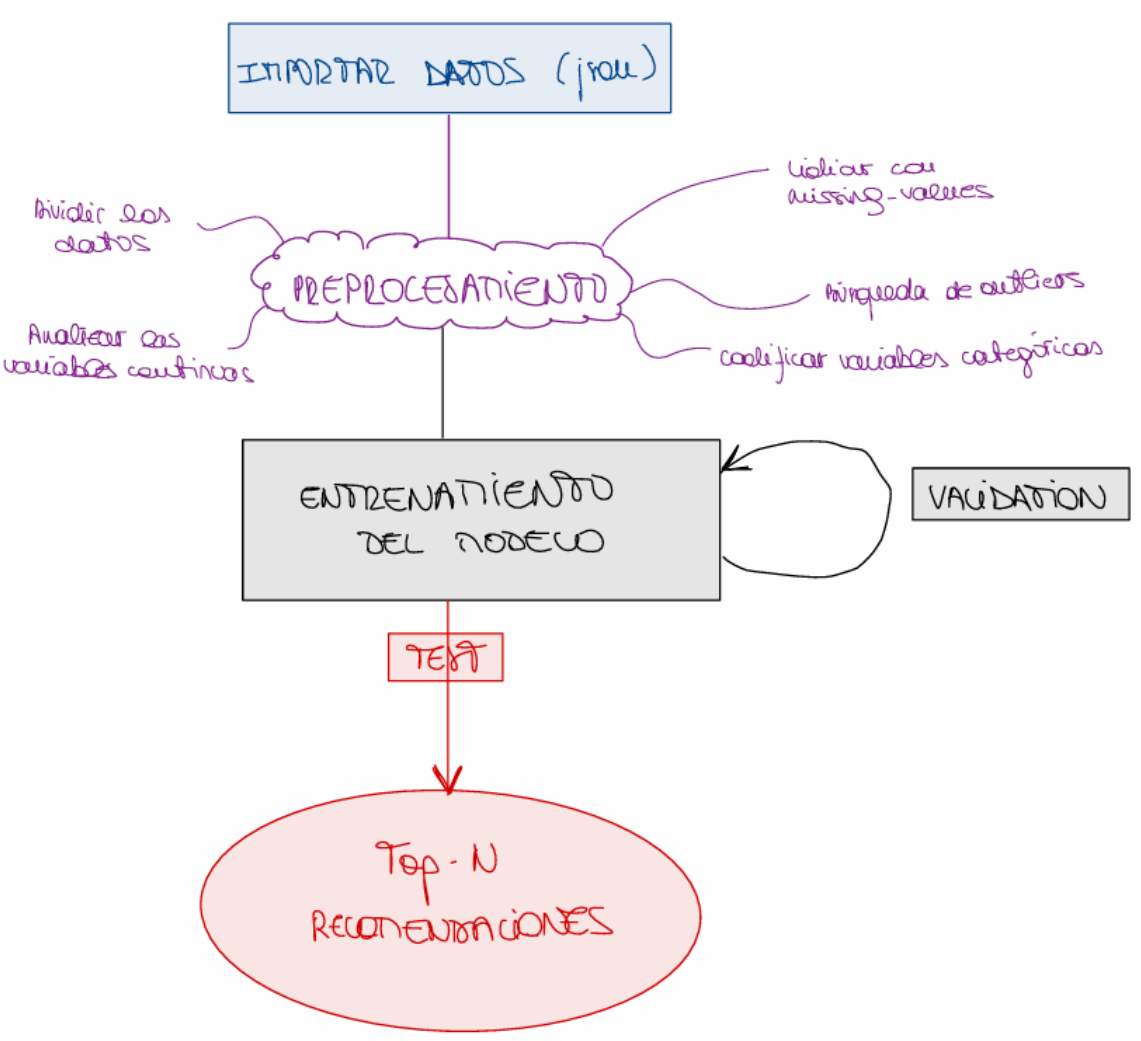


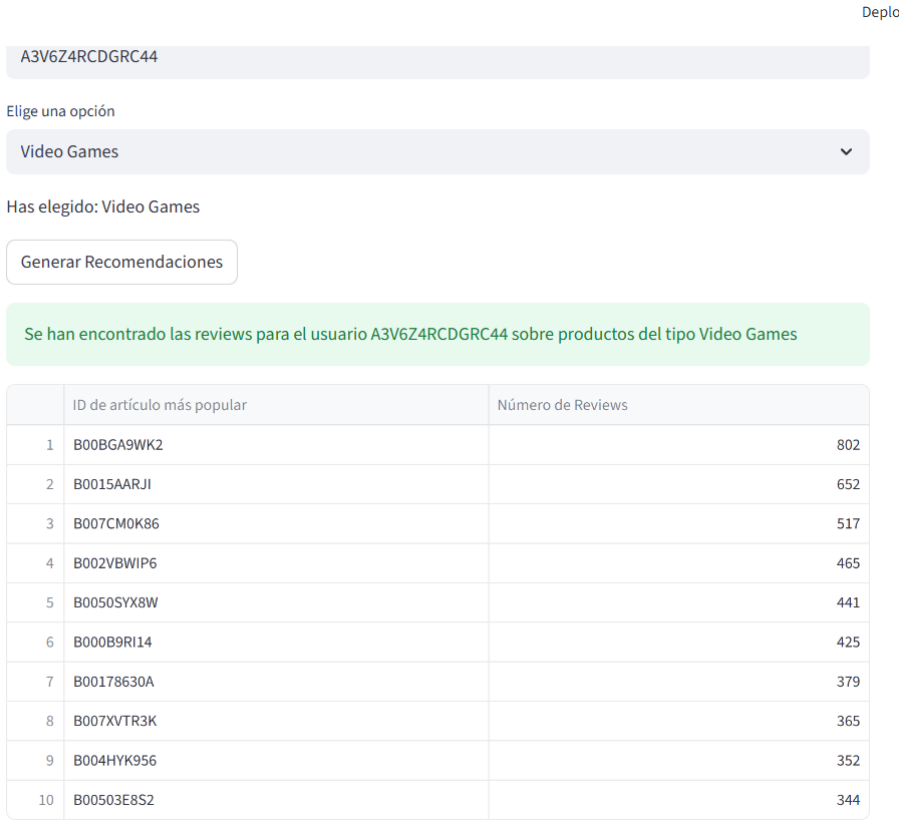
Figura 19. Diagrama que representa el proceso de la creación de un modelo de Machine Learning.

APARTADO 3: Mecanismo identificador de recomendaciones

Por último, para realizar este apartado, como se dejaba bastante abierto el objetivo del mismo y la forma de realizarlo, en nuestro caso hemos querido implementarlo como una consulta al servidor más, la cual hemos representado en el propio aplicativo de streamlit de la segunda parte del proyecto. Es decir, en este caso, la función que resuelve esta parte se encuentra en el fichero menu_visualizacion.py, y explícitamente al final se puede visualizar una función articulos_populares_no_consumidos() que contiene las consultas pertinentes para llevar esta consulta a cabo. Además, lo bueno de haberla introducido en este fichero, es que hemos podido reutilizar funciones anteriormente definidas. Si nos remontamos de nuevo a la página de streamlit, vemos que al pulsar la opción de 'Artículos populares no consumidos' se nos despliega un menú algo diferente, como el que se muestra en la **Figura 20**.

Figura 20. Menú de 'artículos populares no consumidos' en la app de streamlit

En este menú, si nos fijamos, tenemos que introducir de nuevo un id de reviewer y seleccionar una de las categorías para comenzar con la búsqueda y las recomendaciones. En caso de elegir uno, observamos como nos devuelve una tabla (creada mediante un Dataframe de pandas) con los 10 artículos más populares no consumidos por ese reviewer y ordenados de mayor a menor popularidad, como si quisiera incitar al usuario a probar y comprar ese artículo. Realmente este apartado no conlleva una complicación muy alta, pero es cierto que es de las consultas más útiles bajo nuestro punto de vista y que nos ha encantado realizar. Por ejemplo, para el usuario 'A3V6Z4RCDGRC44' y la categoría Video Games obtenemos el resultado de la siguiente imagen:



	ID de artículo más popular	Número de Reviews
1	B00BGA9WK2	802
2	B0015AARJI	652
3	B007CM0K86	517
4	B002VBWIP6	465
5	B0050SYX8W	441
6	B000B9RI14	425
7	B00178630A	379
8	B007XVTR3K	365
9	B004HYK956	352
10	B00503E8S2	344

Figura 21. Tabla de recomendaciones generada para el usuario 'A3V6Z4RCDGRC44' en la categoría de Video Games

Opcional. Séptima parte. Mejora en visualización

Por último, cabe destacar que para llevar a cabo este opcional, como bien se explicó al principio de la Segunda Parte, ha sido fundamental la librería de streamlit, que gracias a todo el contenido aprendido en la asignatura de Visualización de Datos nos ha dado la posibilidad de generar un menú interactivo mucho más práctico, visual e insuperable. Realmente todo ha funcionado con una comunicación constante entre Python y el propio aplicativo en el servidor, pero ha sido de alta utilidad y consideramos que ha mejorado exponencialmente el resultado de nuestro proyecto.

Conclusión

En definitiva, consideramos que este proyecto ha encarecido enormemente nuestros conocimientos, no podemos estar más contentos con el resultado final y sobre todo con los conocimientos ganados a medida que avanzábamos con el desarrollo del mismo. Sin duda ha sido un aprendizaje muy a meno, y nos ha permitido ganar una base mucho más estable en todos los aplicativos empleados en el proyecto.